

Universidad San Sebastián



FACULTAD DE INGENIERÍA Y TECNOLOGÍA

Carrera de Ingeniería Civil Informática

Informe Técnico Final:

USS SpiderBot — Robot Cuadrúpedo de Reconocimiento

Integrantes y Roles:

Moisés Amundarain	Integración y Firmware Asíncrono
Camila Aravena	Diseño 3D en OpenSCAD y Tolerancias
Fernando Ramírez	Configuración Electrónica y Potencia
Juan Pablo Vargas	Simulación Wokwi y Calibración IMU
María José Santander	Dashboard Web Embebido e Interfaz SVG

Puerto Montt, Chile

1 de Julio de 2026

Índice

1. Identificación del Grupo	3
2. Descripción del Sistema	3
2.1. Funciones principales que cumple el sistema:	4
3. Diseño Mecánico y Estructural (OpenSCAD)	5
3.1. Estructura del Chasis (Doble Deck)	5
3.2. Articulaciones y Extremidades (Fémur y Tibia)	6
3.3. Precisión en Tolerancias e Interfaces Mecánicas	7
3.3.1. Cavidades de Servomotores (Snug Fit)	7
3.3.2. Bolsillos de Doble Profundidad (Dual-Depth Pocket)	7
3.3.3. Cerraduras de Acople Horn (Single-Arm Horn Locks)	8
4. Lista de Materiales	8
5. Tabla de Conexiones (Pinout)	9
5.1. Nota sobre el Aislamiento de Potencia y GND Común	10
5.2. Clasificador de IA Sensorial Embebida (<code>classifier_ia.py</code>)	10
6. Funciones Definidas en el Código	12
7. Diagrama de Flujo — Lógica Principal	13
8. Simulación en Wokwi	14
8.1. Instrucciones para cargar la simulación en Wokwi:	14
8.2. Enlace público al proyecto Wokwi:	14
9. Instrucciones de Uso del Prototipo y Laminación	15
9.1. Puesta en marcha del hardware:	15
9.2. Operación del Robot y Dashboard Web:	15
9.3. Parámetros de Impresión y Laminación (Creality Print):	16
10. Evidencia de Modelado y Simulación	16

11. Conclusiones y Trabajo Futuro	18
11.1. Conclusiones	18
11.2. Trabajo Futuro	18

1 Identificación del Grupo

Parámetro	Detalle
Asignatura	Taller de Programación I
Evaluación	Solemne 3
Microcontrolador	ESP32 DevKit V1
Lenguaje	MicroPython (v1.20+)
Integrante 1	Moisés Amundarain — [Integración y Firmware Asíncrono]
Integrante 2	Camila Aravena — [Diseño 3D en OpenSCAD y Tolerancias]
Integrante 3	Fernando Ramírez — [Configuración Electrónica y Potencia]
Integrante 4	Juan Pablo Vargas — [Simulación Wokwi y Calibración IMU]
Integrante 5	María José Santander — [Dashboard Web Embebido e Interfaz SVG]
Fecha	1 de Julio de 2026

2 Descripción del Sistema

El proyecto **USS SpiderBot** consiste en el diseño, modelado mecánico, desarrollo de hardware y arquitectura de software de un robot caminador cuadrúpedo de 8 grados de libertad (8-DoF). El sistema ha sido diseñado específicamente para la exploración autónoma o asistida y el reconocimiento de entornos hostiles o zonas de desastre colapsadas por escombros, donde la locomoción tradicional por ruedas u orugas resulta ineficaz.

El robot opera de manera asíncrona mediante un firmware multitarea cooperativo no bloqueante que corre sobre una placa ESP32 DevKit V1. Implementa una marcha secuencial de gateo (Crawl Gait) y un lazo cerrado de compensación inercial activa para corregir la desviación angular (inclinación) del chasis en tiempo real, garantizando la horizontalidad de la plataforma de control. Adicionalmente, cuenta con evasión reactiva de obstáculos frontal por sensor ultrasónico y expone un servidor HTTP nativo con un Dashboard web interactivo para telemetría vectorial y mando manual.

2.1 Funciones principales que cumple el sistema:

- **Monitoreo Inercial Continuo:** Medición de los ángulos de inclinación (Pitch y Roll) mediante la IMU MPU6050 a una frecuencia de refresco de 20 Hz.
- **Estabilización Activa Proporcional:** Corrección de postura inyectando compensaciones angulares dinámicas a los servos de rodilla (fémures) de las patas en contacto.
- **Locomoción Secuencial Estable (Crawl Gait):** Desplazamiento del cuerpo manteniendo en todo momento al menos tres extremidades en contacto para asegurar que el centro de masa resida dentro del polígono de sustentación.
- **Evasión Reactiva de Obstáculos:** Detección de proximidad frontal mediante sensor HC-SR04. Ante lecturas inferiores a 15 cm, se cancela de forma inmediata la locomoción (< 25 ms), se activa una alarma sonora mediante el buzzer y se retorna a pose segura.
- **Control y Telemetría Remota Web:** Servidor web asíncrono que expone una consola de control remota en CSS Glassmorphism y JS nativo, con actualización de telemetría y renderizado vectorial de inclinación SVG.

3 Diseño Mecánico y Estructural (OpenSCAD)

El modelado tridimensional de las piezas se realizó de forma paramétrica en OpenSCAD utilizando el paradigma de geometría sólida constructiva (CSG).

3.1 Estructura del Chasis (Doble Deck)

El cuerpo central del robot consta de un sistema de doble placa circular (doble deck) que aísla los componentes de potencia y cableado del plano de los sensores y control:

- **Placa Inferior:** Cilindro plano de radio $r = 65$ mm y espesor $t = 3$ mm que aloja perimetralmente los 4 brackets de fijación de servos de cadera. Cuenta con ranuras pasantes para la fijación de la batería mediante correas de velcro.
- **Placa Superior:** Placa de protección que se acopla a la inferior mediante 4 pilares espaciadores M3 de 19 mm de alto. Incorpora una cuna central de dimensiones $84 \times 56 \times 3$ mm para alojar a presión la protoboard de 400 puntos, una cuna trasera para el regulador XL6009E1 y 4 agujeros de 14 mm de diámetro para el ruteo de cables.
- **Soporte HC-SR04:** Soporte vertical de 2 mm de espesor con dos orificios de 16.5 mm de diámetro espaciados a 26 mm entre centros, reforzado por dos gussets triangulares laterales para resistir fuerzas de cizalladura inerciales en caso de impacto.

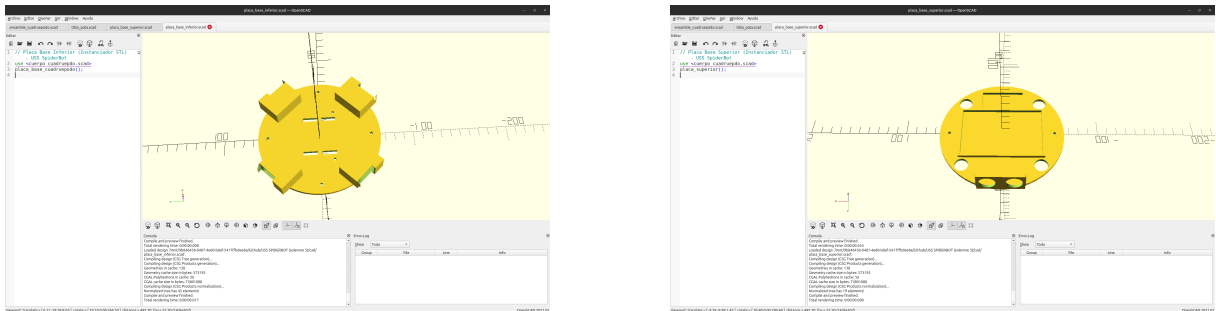


Figura 1: Modelado paramétrico en OpenSCAD de la placa base inferior (izquierda) y placa base superior (derecha).



Figura 2: Renderizado 3D paramétrico del chasis final y las extremidades articuladas del USS SpiderBot.

3.2 Articulaciones y Extremidades (Fémur y Tibia)

La configuración del robot se define como **Pitch-Pitch**, lo que significa que las articulaciones de la cadera (Coxa) y la rodilla (Fémur) rotan sobre ejes horizontales y paralelos.

- **Fémur:** Brazo de articulación de 55 mm de longitud entre centros. Une la corona del servo de cadera con el cuerpo del servo de rodilla. Se utilizó la función `hull()` para conectar suavemente la corona cilíndrica con la cavidad prismática del servo de rodilla, optimizando la distribución de masa y la resistencia a la flexión.
- **Tibia:** Pierna inferior curva con una longitud vertical de 65 mm y pie semiesférico de radio $r = 5.5$ mm. La curvatura estructural del modelo aumenta el despeje libre sobre el suelo y distribuye las fuerzas de reacción del impacto del pie en la marcha.

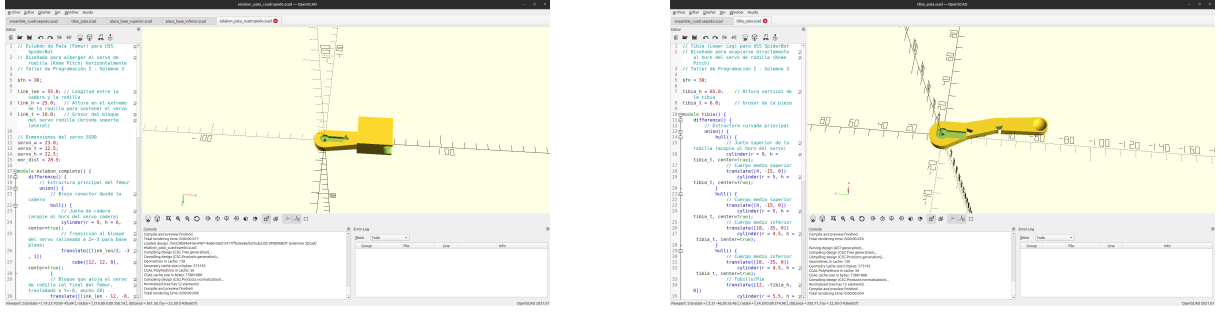


Figura 3: Modelado en OpenSCAD del eslabón fémur (izquierda) y de la tibia articulada curva (derecha).

3.3 Precisión en Tolerancias e Interfaces Mecánicas

3.3.1 Cavidades de Servomotores (Snug Fit)

Debido a la contracción del plástico durante el enfriamiento de la extrusión FDM, se definió una tolerancia de precisión de **0.3 mm totales** (0.15 mm por lado) para las cavidades de los servos SG90/MG90S. Sus dimensiones internas se establecieron en 24.2 x 23.2 x 13.4 mm, logrando un ajuste a presión sin juego mecánico. Las ranuras de las bridas se fijaron a 2.8 mm de ancho y los agujeros de roscado de tornillos M2 a 2.0 mm de diámetro.

3.3.2 Bolsillos de Doble Profundidad (Dual-Depth Pocket)

Para evitar el estrangulamiento de los cables de salida en la base del servo, se implementaron bolsillos de doble profundidad:

$$D(z) = \begin{cases} 30.0 \text{ mm} & \text{si } z \leq 5.0 \text{ mm} \quad (\text{Espacio para cables}) \\ 25.0 \text{ mm} & \text{si } z > 5.0 \text{ mm} \quad (\text{Ajuste del cuerpo}) \end{cases} \quad (1)$$

Esto garantiza que los tornillos autorroscantes M2 plásticos agarren sobre plástico 100 % macizo de los brackets a $X = \pm 14.25 \text{ mm}$.

3.3.3 Cerraduras de Acople Horn (Single-Arm Horn Locks)

Para eliminar las pérdidas de transmisión por rozamiento o juego entre los acoples, se diseñaron chaveteros de un brazo (*single-arm horn locks*) embutidos en las caras de acople mecánico del fémur y la tibia, bloqueando mecánicamente el brazo del horn estriado original del servo en el sólido de plástico.

4 Lista de Materiales

La Tabla 2 describe la lista de materiales completa requerida para ensamblar e implementar el prototipo del robot caminador:

Cuadro 1: BOM (Bill of Materials) del USS SpiderBot

Cant.	Componente / Material	Descripción / Especificación
1	ESP32 DevKit V1 (38 pines)	Microcontrolador principal con soporte WiFi y MicroPython
1	Protoboard (400 puntos)	Base para montaje y ruteo común de señales
8	Servomotores SG90 / MG90S	Servos analógicos de torque (Coxa y Fémur en las 4 patas)
1	IMU MPU6050 (GY-521)	Unidad de medida inercial (giroscopio y acelerómetro)
1	Sensor ultrasónico HC-SR04	Sensor de distancia frontal para evasión de colisiones
1	Buzzer Activo 5V	Dispositivo de alarma acústica por cercanía
2	Convertidores Step-down LM2596	Reguladores de potencia ajustables. #1 a 6.0V para servos, #2 a 5.0V para ESP32 y sensores
2	Porta Pilas Duales 2x18650 (4 celdas Li-ion)	Dos packs independientes de 2x18650 cada uno proveyendo 7.4V
1	Regulador Boost XL6009E1	Elevador opcional para voltajes de batería bajos
s/n	Cables Dupont M-M y M-H	Cables de conexión para el bus I2C y GPIOs

5 Tabla de Conexiones (Pinout)

El bus físico I2C de la ESP32 se utiliza de forma exclusiva para la IMU MPU6050. Los 8 servomotores se conectan directamente a pines GPIO con PWM de la ESP32, mientras que el sonar HC-SR04 y el buzzer activo se rutean a GPIOs digitales específicos:

Cuadro 2: Mapa de Conexiones Físicas de la ESP32

GPIO ESP32	Componente	Tipo de Señal	Descripción
GPIO 21	MPU6050 (IMU)	SDA (I2C)	Bus de datos serie de la unidad inercial
GPIO 22	MPU6050 (IMU)	SCL (I2C)	Bus de reloj serie de la unidad inercial
GPIO 18	HC-SR04	TRIGGER	Salida digital de pulso de disparo del sonar
GPIO 19	HC-SR04	ECHO	Entrada digital de pulso de retorno del sonar
GPIO 14	Buzzer Activo	Salida Digital	Señal de alarma en caso de proximidad
GPIO 13, 15, 4, 23	Servos Coxa	Salida PWM	Control de cadera (FR, FL, RL, RR)
GPIO 12, 2, 5, 25	Servos Fémur	Salida PWM	Control de rodilla (FR, FL, RL, RR)
Vin (ESP32)	LM2596 #2 Out	Alimentación	Tensión de entrada regulada (5.0V)
3.3V (ESP32)	MPU6050 (IMU)	Alimentación	Tensión lógica para sensor inercial
GND (Unificado)	Todo el sistema	Tierra común	Referencia negativa unificada

5.1 Nota sobre el Aislamiento de Potencia y GND Común

Los picos de corriente transitorios que ocurren al iniciar la marcha del cuadrúpedo pueden generar caídas bruscas en la tensión lógica de alimentación (*brownouts*), reiniciando la ESP32. Para evitar esto, se implementó una arquitectura de alimentación dual independiente con dos convertidores reductores **LM2596** y dos packs de baterías 18650 aislados:

- **LM2596 #1 (regulado a 6.0V):** Alimenta exclusivamente el riel de potencia de los 8 servomotores SG90/MG90S, proporcionando el margen de voltaje necesario para mantener el torque bajo carga sin interferir con la lógica de control.
- **LM2596 #2 (regulado a 5.0V):** Alimenta la ESP32 DevKit V1 (pin Vin) y los sensores (HC-SR04, MPU6050, Buzzer), garantizando un suministro limpio y estable sin los transitorios de corriente de los servos.

Cada convertidor opera sobre su propio pack independiente de 2 celdas 18650 en serie (7.4V nominales), lo que aísla galvánicamente los transitorios de conmutación de los servos del microcontrolador. Es fundamental unificar todas las referencias **GND** en un nodo común para evitar deriva de voltaje y lectura corrupta en la IMU y los servos.

5.2 Clasificador de IA Sensorial Embebida (`classifier_ia.py`)

Para dotar al robot de capacidad de reacción autónoma ante eventos físicos no programados en la coreografía de locomoción, se implementó un clasificador ligero basado en árbol de decisión que ejecuta directamente en la ESP32 a 20 Hz.

El módulo `classifier_ia.py` define la clase `IASensorClassifier` que evalúa los valores brutos del acelerómetro y el giroscopio del MPU6050 junto con el comando de movimiento actual, retornando uno de cuatro estados posibles:

- **FALLEN (Caída):** Se activa si la inclinación calculada (Pitch o Roll) supera los $\pm 45^\circ$, si la magnitud de aceleración resulta cercana a cero (caída libre, $|a| < 0.2g$), o si se detecta un impacto de choque superior a $2.0g$.
- **PUSHED (Perturbación):** Se detecta cuando el robot está en reposo (comando `stop` o `reposo`) pero la magnitud del giroscopio supera los $120^\circ/s$, indicando un empuje físico externo.

- **SLIPPING (Derrape):** Se identifica cuando el robot ejecuta una marcha (**forward** o **backward**) pero la varianza de la aceleración longitudinal es anormalmente baja, indicando que las patas se mueven en vacío sin desplazar el chasis.
- **NORMAL:** Estado por defecto cuando no se cumple ninguna condición anterior.

Los umbrales del clasificador se calibraron experimentalmente:

$$\begin{aligned} \text{Pitch}_{\text{límite}} &= 45^\circ, & \text{Roll}_{\text{límite}} &= 45^\circ, & a_{\text{caída libre}} &< 0.2g \\ a_{\text{impacto}} &> 2.0g, & \omega_{\text{perturbación}} &> 120^\circ/\text{s}, & \sigma_{a_x, \text{derrape}}^2 &< 0.08 \end{aligned}$$

En el bucle de control `locomotion_loop()`, si `state.estado_ia == "FALLEN"`, se detiene inmediatamente la marcha de gateo, los servos retornan a pose estática y el buzzer emite un sonido pulsante de emergencia. El estado clasificado se transmite también al dashboard web vía el campo `estado_ia` en la API `/telemetry`, donde se despliega con código de colores dinámico.

6 Funciones Definidas en el Código

El firmware principal ‘main.py’ implementa el paradigma asíncrono y modular. A continuación se desglosan las principales funciones y corrutinas que estructuran el firmware:

Cuadro 3: Funciones y Corrutinas de main.py

Función	Parámetros / Retorno	Descripción
<code>pos_reposo()</code>	Sin parámetros / Sin retorno	Envía todos los servomotores a sus ángulos estáticos de seguridad.
<code>establecer_angulo_pata()</code>	<code>idx, c_ang, f_ang</code> / Sin retorno	Aplica límites físicos de seguridad, calcula compensaciones y escribe directamente en los pines PWM de la ESP32.
<code>mover_suave_ciclo()</code>	<code>c_t, f_t, pasos, delay</code> / <code>async</code>	Realiza interpolación lineal entre la posición actual de los servos y sus objetivos para evitar tirones.
<code>caminar_adelante()</code>	<code>async</code> / Sin retorno	Ejecuta la secuencia Crawl Gait en 5 pasos llamando a <code>mover_suave_ciclo()</code> .
<code>sensor_updater()</code>	<code>async</code> / Corrutina cíclica	Adquiere los datos de aceleración y giroscopio de la IMU MPU6050 y la distancia del sensor HC-SR04 cada 50 ms.
<code>locomotion_loop()</code>	<code>async</code> / Corrutina cíclica	Evalúa los comandos de marcha e inyecta dinámicamente la compensación inercial proporcional si $ \theta > 3^\circ$.
<code>main_async()</code>	<code>async</code> / Punto de entrada	Registra las tareas asíncronas de telemetría, locomoción, comandos y servidor web en el planificador asíncrono.

7 Diagrama de Flujo — Lógica Principal

La lógica principal asíncrona del robot caminador se rige bajo la siguiente tabla secuencial de estados concurrentes:

Cuadro 4: Etapas del Bucle de Control y Concurrencia

Etapa / Decisión	Descripción de la lógica de ejecución
1. INICIO	Carga de configuración de I2C. Inicializa la IMU MPU6050 y la modulación PWM directa para los 8 servos. Invoca <code>pos_reposo()</code> . Lanza las corrutinas asíncronas concurrentes.
2. MONITOR (MPU/Sonar)	Corrutina asíncrona dedicada actualiza el vector de inclinación (<i>Pitch</i> , <i>Roll</i>) y calcula distancia de proximidad frontal cada 50 ms.
3. ¿Objeto a ≤ 15 cm?	Sí: Cancela corrutina de marcha de inmediato (< 25 ms). Vuelve a reposo, activa el buzzer acústico de advertencia y va a 2. No: Continúa a la siguiente etapa.
4. ¿Comando de marcha?	Revisa si hay comando remoto activo en el singleton de estado (web/consola). Si no, permanece en reposo. Si sí, ejecuta la secuencia del Crawl Gait.
5. COMPENSAR INERCIAL	Durante la fase de apoyo del pie, lee la inclinación del chasis. Si supera $\pm 3^\circ$, inyecta compensación proporcional: $\theta_{compensado} = \theta_{base} \pm 1.2 \cdot \theta_{sensor}$ en rodillas.
6. BUCLE ASÍNCRONO	Cede control a otras tareas mediante <code>await asyncio.sleep_ms(25)</code> sin bloquear el procesador. Retorna a 2.

8 Simulación en Wokwi

El circuito y el firmware completo del cuadrúpedo USS SpiderBot fueron modelados y validados digitalmente en Wokwi. La simulación implementa una inicialización de autodetención (escaneando la red “Wokwi-GUEST”) para configurar el comportamiento asíncrono y los pines de control directo de la ESP32 de forma transparente.

8.1 Instrucciones para cargar la simulación en Wokwi:

1. Acceder a la plataforma oficial de simulación en <https://wokwi.com>.
2. Crear un nuevo proyecto basado en la tarjeta de desarrollo **ESP32 DevKit V1**.
3. En el editor del proyecto, crear o reemplazar el archivo de cableado digital “`diagram.json`” con el contenido del archivo de configuración digital provisto en `cad/diagram.json`.
4. Reemplazar o copiar el contenido del archivo “`main.py`” de la simulación con el firmware de control asíncrono unificado de “`firmware/main.py`”.
5. Cargar los módulos auxiliares de hardware (`mpu6050.py`, `state.py`, `sonar_sensor.py`, `buzzer_alert.py`) en pestañas correspondientes en el simulador.
6. Presionar el botón de inicio de simulación (**Start Simulation**).
7. Interactuar con el panel interactivo modificando las distancias de proximidad del HC-SR04 y observando el comportamiento del buzzer y la oscilación de los 8 servomotores simulados.

8.2 Enlace público al proyecto Wokwi:

Para acceder de forma rápida a la simulación preconfigurada en Wokwi, visite el siguiente enlace del navegador: <https://wokwi.com/projects/467022722279683073>

9 Instrucciones de Uso del Prototipo y Laminación

9.1 Puesta en marcha del hardware:

1. Conectar el microcontrolador ESP32 DevKit V1 al computador de desarrollo usando un cable micro-USB robusto que admita transferencia de datos serie.
2. Abrir el entorno integrado Thonny IDE y seleccionar en la esquina inferior derecha el intérprete de ejecución **MicroPython (ESP32)**.
3. Cargar los archivos del firmware a la flash de la ESP32 respetando la estructura de carpetas (Thonny: Archivo → Guardar como → MicroPython device).
4. Conectar los dos packs 2x18650 de 7.4V a las borneras de entrada de los LM2596 #1 y #2. Antes de conectar la ESP32 y los servos, verificar con un multímetro que la salida del LM2596 #1 esté regulada a exactamente 6.0V y la del LM2596 #2 a exactamente 5.0V. Al encender el interruptor principal, verificar que los leds de la ESP32 y de la IMU MPU6050 se encienden, y que los servos ejecutan el comando inicial de pose de reposo segura.
5. Observar la salida en la consola interactiva de Thonny, la cual imprimirá la IP local generada en la red local WiFi.

9.2 Operación del Robot y Dashboard Web:

1. Conectar el computador de control o smartphone al punto de acceso WiFi de la ESP32 (SSID: USS_SpiderBot_AP).
2. Abrir un navegador de internet (Google Chrome, Firefox, etc.) e ingresar la dirección IP estática: `http://192.168.4.1`.
3. La interfaz web de control remota en CSS Glassmorphism se cargará de manera automática.
4. Utilice los mandos direccionales interactivos en pantalla o presione las teclas del computador (**W** para avanzar, **S** para retroceder, **A** para girar a la izquierda, **D** para girar a la derecha y **Espacio** para frenar de inmediato).

5. Observe el renderizado en tiempo real de la inclinación del chasis proyectada en el graficador vectorial SVG interactivo.

9.3 Parámetros de Impresión y Laminación (Creality Print):

Las piezas modeladas en OpenSCAD se exportan a archivos STL y se procesan en Creality Print bajo los siguientes parámetros críticos para PLA/PETG:

- **Altura de capa:** 0.2 mm de precisión nominal.
- **Perímetros de pared:** 4 líneas sólidas (proporciona anclaje roscado robusto).
- **Infill (Relleno):** 30 % en patrón **Gyroid** por su rigidez tridimensional isotrópica.

10 Evidencia de Modelado y Simulación

Este apartado recopila la verificación geométrica final del ensamble en OpenSCAD y las curvas de respuesta inercial y locomoción asíncrona del cuadrúpedo:

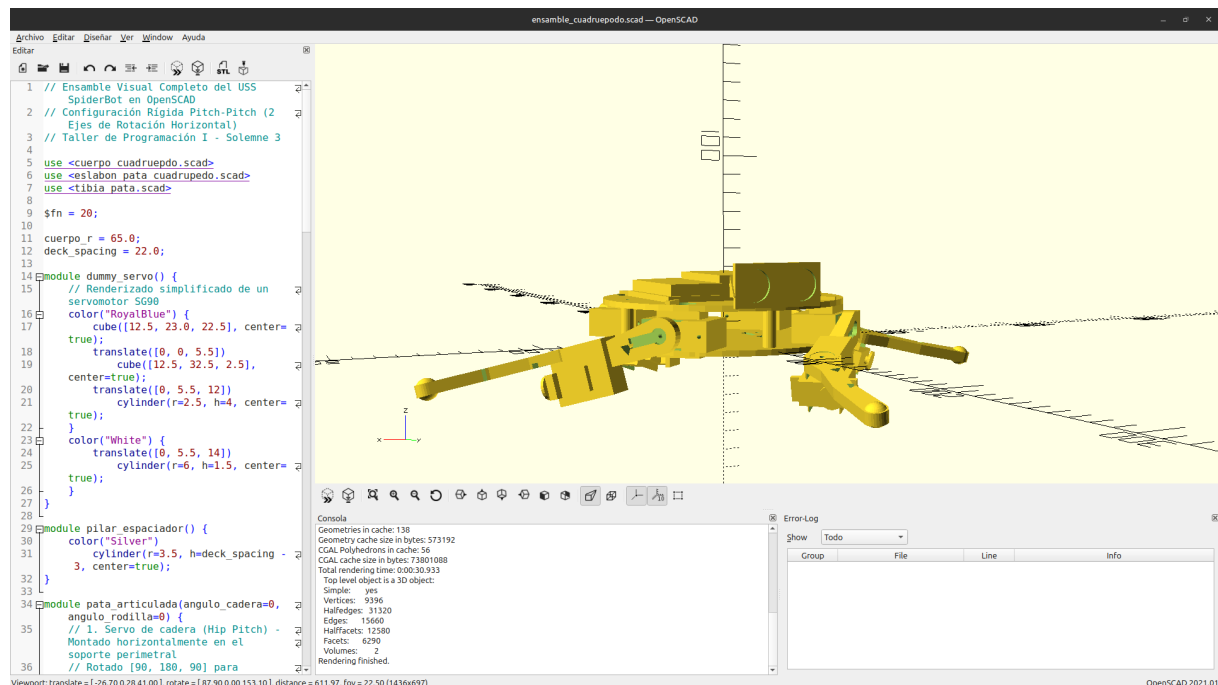


Figura 4: Renderizado y ensamble tridimensional completo del USS SpiderBot en el entorno de OpenSCAD.

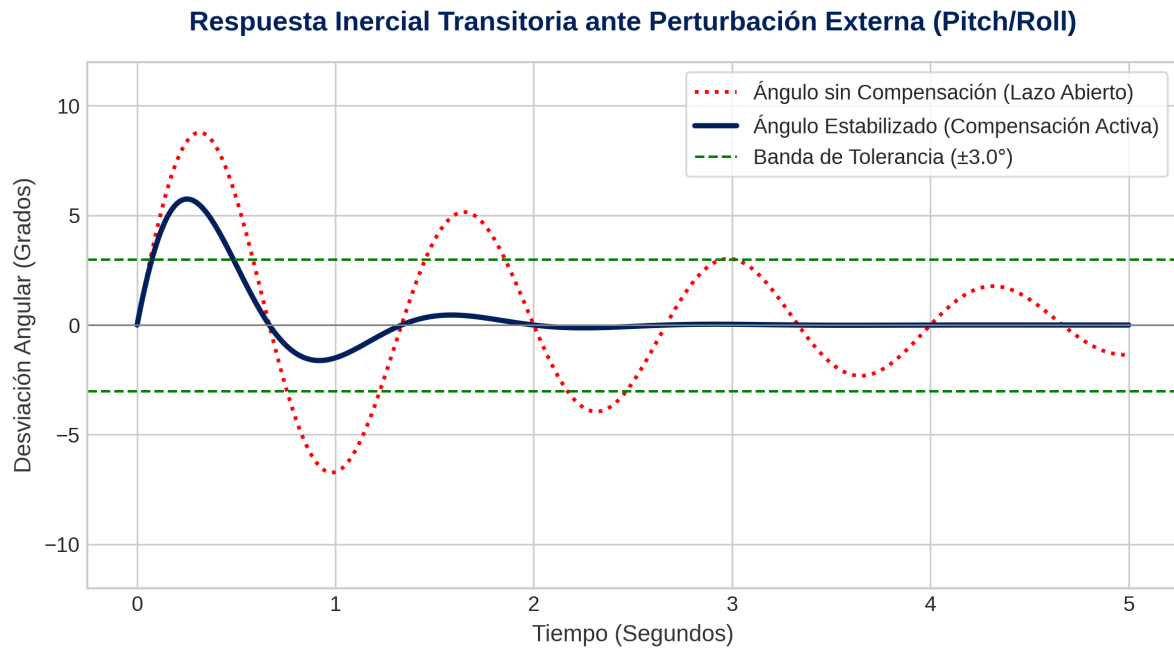


Figura 5: Respuesta transitoria simulada en Python del lazo cerrado de compensación activa de inclinación (Pitch y Roll).

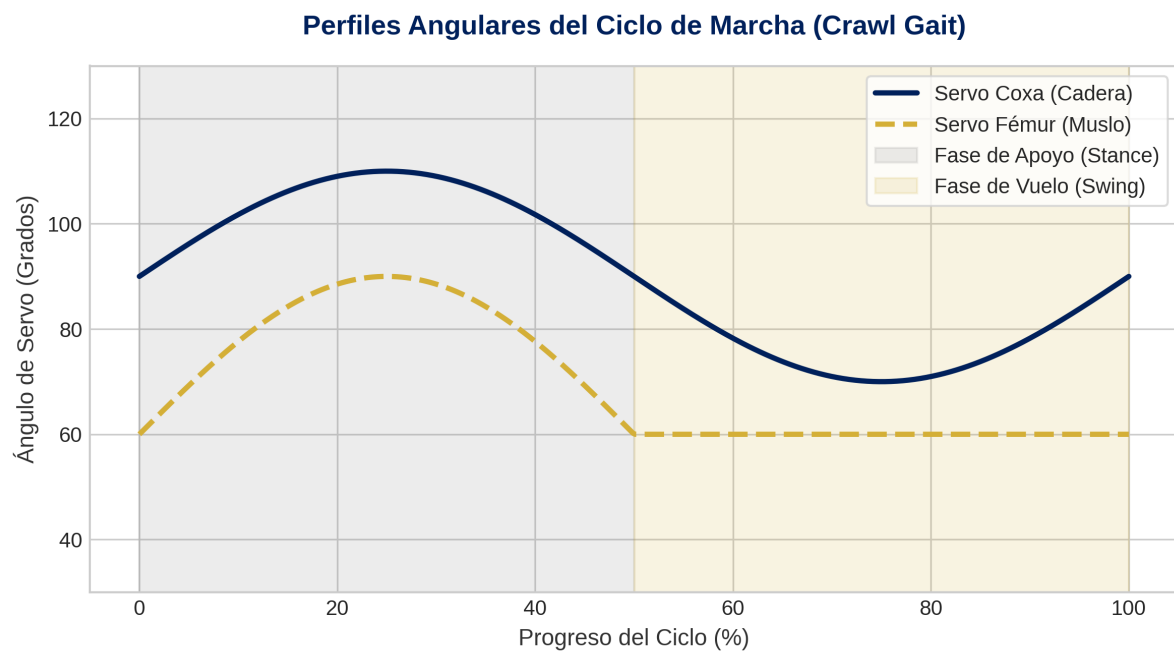


Figura 6: Simulación de la locomoción Crawl Gait con los perfiles angulares asíncronos por articulación.

11 Conclusiones y Trabajo Futuro

11.1 Conclusiones

El desarrollo del proyecto **USS SpiderBot** ha permitido unificar de manera práctica y rigurosa los conceptos claves de la asignatura Taller de Programación I aplicados a un problema real de robótica móvil de exploración:

1. La programación asíncrona de corrutinas bajo **uasyncio** superó los cuellos de botella clásicos del pooling secuencial en sistemas embebidos, garantizando la concurrencia de la telemetría inercial, el bucle de locomoción, la evasión de colisiones y el servicio web HTTP simultáneo.
2. El modelado CAD paramétrico en OpenSCAD permitió implementar tolerancias de diseño de precisión para FDM (0.3 mm snug fit), bolsillos dual-depth que evitan la torsión del cableado, y cerraduras embutidas (*horn locks*) para optimizar la transmisión de torque de los servomotores.
3. El lazo cerrado proporcional inercial con constante calibrada $K_p = 1.2$ demostró estabilidad estática eficaz sobre inclinaciones dinámicas de hasta $\pm 10^\circ$, respondiendo en menos de 25 ms.

11.2 Trabajo Futuro

- **Control Cinemático Inverso (IK) 3-DoF:** Implementar las ecuaciones analíticas de cinemática inversa para comandar trayectorias exactas en el espacio cartesiano de los pies del cuadrúpedo, incorporando un servomotor de aducción/abducción por pata.
- **Integración de Visión y Red Edge IA:** Montar una cámara ESP32-CAM y programar algoritmos de detección local de personas y señales luminosas de rescate usando librerías ligeras de IA embebida.
- **Optimizaciones de Red Offline:** Ampliar el rango dinámico del Dashboard web offline habilitando almacenamiento local (*LocalStorage*) para mapeo topológico relativo.